INSIDE STLC ACADEMY

API TESTING MASTERCLASS

Postman Collections Guide

Organise, parameterise and run professional Postman collections.

Postman is the tool most testers reach for first, but a folder full of one-off requests is not a test asset. This guide shows you how to build a collection that is organised, parameterised and repeatable — something you can hand to a colleague, run in CI, and be proud to put in your portfolio.

What a Collection Is

A collection is a saved, shareable group of API requests. It bundles the requests themselves, the folder structure that organises them, environment and variable definitions, pre-request scripts, and the tests (assertions) that run after each response. Think of it as an executable test suite for an API rather than a scratchpad.

Folder Structure

Mirror the API or the user journeys, not your trial-and-error history. A clear structure makes the collection self-documenting.

- Top level: one folder per resource or feature (e.g. Auth, Users, Orders).
- Within a folder: group by operation or scenario (Create, Retrieve, Update, Delete; happy path vs negative).
- Name requests by intent — "Create user (valid)" and "Create user (missing email → 400)", not "POST 2".
- Keep a Setup/Teardown folder for login and cleanup requests that other folders depend on.

Variables & Environments

Never hard-code a base URL, token or ID into a request. Store them as variables and let scope decide where each value lives. Postman resolves a variable from the narrowest scope outwards.

Scope	Lives in	Use for	Example
Global	The whole workspace	Rarely-changing values shared everywhere	apiVersion
Collection	One collection	Values true for every environment	defaultPageSize
Environment	A selected environment (Dev/Staging/Prod)	Values that differ per environment	baseUrl, authToken
Local / Data	A single run or data file row	Iteration-specific data	username, expectedStatus

Reference a variable with double curly braces, e.g. a request URL of {{baseUrl}}/users/{{userId}}. Switch the active environment (top-right dropdown) to point the same collection at Dev, Staging or Prod without editing a single request. Store secrets as environment "secret" variables so they are masked.

Writing Tests in the Tests Tab

The Tests tab runs JavaScript after the response arrives. Each pm.test block is one assertion that shows as pass or fail in the run summary.

- pm.test("Status is 200", () => pm.response.to.have.status(200));
- pm.test("Response is JSON", () => pm.response.to.be.json);
- const body = pm.response.json(); pm.test("Has id", () => pm.expect(body.id).to.exist);
- pm.test("Responds within 800ms", () => pm.expect(pm.response.responseTime).to.be.below(800));

Chaining Requests with Variables

Real API tests are sequences: create a resource, then read, update and delete it. Capture a value from one response and reuse it in the next by saving it to a variable.

1. In the first request (e.g. Login), open the Tests tab.
2. Read the value from the response: const token = pm.response.json().access_token;
3. Save it to an environment variable: pm.environment.set("authToken", token);
4. In later requests, use it: set the Authorization header to Bearer {{authToken}}.
5. Repeat the pattern for IDs — pm.collectionVariables.set("userId", pm.response.json().id) after a create, then GET {{baseUrl}}/users/{{userId}}.

Running a Collection

Use the Collection Runner (Run button on the collection) to execute every request in order and see a pass/fail summary. To run the same requests against many inputs, attach a data file.

- Prepare a CSV or JSON file where each row/object is one iteration; column names become variables.
- In the Runner, select the collection, choose the environment, and upload the data file.
- Reference the columns as variables in requests and tests (e.g. {{username}}, {{expectedStatus}}).
- For unattended runs, the Newman CLI executes the exported collection in CI — newman run collection.json -e env.json -d data.csv.

Exporting & Sharing

- Export the collection (and each environment) as v2.1 JSON for version control or Newman.
- Never commit real environment secrets — export environments with secret values cleared, or keep them in a separate untracked file.
- Prefer sharing via a workspace or Git-backed collection over emailing JSON, so everyone stays in sync.

Common Mistakes

- Hard-coding base URLs, tokens or IDs into requests instead of using variables.
- Storing secrets at collection or global scope, then exporting and leaking them.
- One giant flat list of requests with no folders and names like "Copy of POST".
- No tests in the Tests tab — a green 200 in the UI is not an assertion.
- Requests that only pass when run manually in a specific order, with no variable chaining to enforce it.

> **INSIDE STLC PRO TIP**

> Build the collection so a stranger could run it: sensible folders, every value a variable, an environment per stage, and at least one assertion per request. A clean, runnable Postman collection exported to your GitHub is a genuine portfolio piece.